

Technical Report: Backend Architecture & API Serving Layer

Project: India Air Quality & Climate Digital Twin

Document Series: Capstone Report Materials — Part 2

Author: Geospatial AI & Full-Stack Engineering Team

Date: October 2026

1. Architectural Overview

The backend of the **India Air Quality & Climate Digital Twin** is built on **FastAPI**, an asynchronous, high-throughput web framework utilizing Python type hints, Pydantic schema validation, and OpenAPI 3.0 auto-generated specifications. It coordinates model inference, geospatial coordinate projections, time series extractions, and counterfactual scenario simulations.



2. API Endpoints Specification

2.1 Health & Diagnostics

- **Route:** GET `/api/v1/health`

- **Purpose:** Service liveness, metadata introspection, and model artifact verification.

- **Sample Response:**

```
{
  "status": "healthy",
  "version": "1.0.0",
  "timestamp": "2026-10-06T01:45:00.000",
  "domain": "all_india_coarse",
  "grid_resolution_deg": 0.25,
  "model_loaded": true
}
```

2.2 Surface AQI & Meteorology Estimation

- **Route:** GET /api/v1/aqi
- **Query Parameters:**
 - lat (float, e.g., 28.61): Latitude coordinate.
 - lon (float, e.g., 77.23): Longitude coordinate.
 - date (string, e.g., "2023-11-05"): Observation date in YYYY-MM-DD.
- **Response Model (AQIResponse):** Returns estimated surface **PM_{2.5}** (**1mu g/m³**), computed CPCB AQI, category name (e.g., "Severe"), category hex color, and collocated meteorological features (**2m** temperature, relative humidity, wind speed).

2.3 Formaldehyde (HCHO) Hotspot Detection

- **Route:** GET /api/v1/hcho-hotspots
- **Query Parameters:**
 - date (string): Date for satellite column evaluation.
 - z_threshold (float, default: 2.0): Anomaly detection Z-score cut-off.
- **Response Format:** Returns total count of identified hotspot pixels, DBSCAN cluster centroids, spatial bounding boxes, and classified source emission types (Industrial/Urban, Biomass Burning, Biogenic VOC).

2.4 Multi-Horizon Forecasting

- **Route:** POST /api/v1/forecast
- **Request Body (ForecastRequest):**

```
{
  "target": "pm25",
  "horizon_days": 7,
  "latitude": 28.61,
  "longitude": 77.23
}
```

- **Execution:** Retrieves historical point series from the coupled state store, passes the sequence through the forecasting engine, and returns predicted trajectories with evaluation metrics (RMSE, MAE, Skill Score).

2.5 Compound Climate Risk Assessment

- **Route:** GET /api/v1/risk
- **Query Parameters:** date (string)

- **Response Format:** Returns subcontinent-wide summary metrics for heat risk, drought risk, air quality exceedance risk, composite risk, and a categorical risk distribution breakdown.

2.6 Real-Time Counterfactual Scenario Simulation

- **Route:** POST /api/v1/scenario/simulate

- **Request Body** (ScenarioRequest):

```
{
  "date": "2023-11-05",
  "delta_temp_c": 2.0,
  "delta_precip_pct": -15.0,
  "delta_fire_pct": -50.0,
  "delta_emissions_pct": -20.0
}
```

- **Response Format:** Returns baseline vs. perturbed subcontinent averages, net **\Delta PM_{2.5}**, net **\Delta AQI**, and counts of severe air quality cells under the proposed intervention.

3. Data Pipelines & High-Performance Storage

1. Analytical Data Formats:

- Datasets are stored in **Apache Parquet** with Snappy compression (data/processed/model1_train_dataset.parquet), providing up to 10× compression ratios and sub-second columnar query response times.

2. Model Persistence:

- Production champion models are serialized using joblib (data/processed/models/best_model1.joblib).

3. In-Memory Caching & Lazy Loading:

- The state store initializes coordinate grids and static tables into memory during startup to eliminate redundant disk reads during request handling.

4. Serving Commands & Operational Verification

Launch FastAPI with Uvicorn worker:

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8000 --workers 2
```

- **Swagger Documentation:** <http://localhost:8000/docs>
- **ReDoc Documentation:** <http://localhost:8000/redoc>
- **Unit Test Coverage:** Verified via `pytest tests/test_api.py -v` (5/5 tests passing).